

TennisAI: Predicción Probabilística de Resultados ATP mediante Gradient Boosting y Ratings ELO

Modelo XGBoost calibrado con ELO dinámico, Rolling Stats y explicabilidad semántica vía LLM

Juan Miguel Cuao Ríos
jmcuaor@eafit.edu.co

Julian Osorio Alturo

Josorioa3@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería · Universidad EAFIT

Curso: Inteligencia Artificial · Semestre 2026-1

Entrega: 19–22 de mayo de 2026

Repositorio: https://github.com/jmcuaor1/_Atp

Resumen

Contexto: La predicción de resultados en el tenis ATP enfrenta retos de volatilidad del rendimiento, especialización por superficie e incertidumbre probabilística. **Objetivo:** Desarrollar un sistema de clasificación binaria que prediga el ganador de un partido ATP con mayor precisión que el baseline basado en rankings oficiales. **Método:** XGBoost con calibración de Platt (CalibratedClassifierCV), enriquecido con ELO dinámico ($K=32$), Rolling Stats (ventana=10) y symmetrization para eliminar sesgo posicional. El sistema incluye API FastAPI, frontend

Next.js y explicabilidad semántica vía LLM. **Resultados:** Accuracy = 71.2%, Brier Score = 0.191, superando el baseline de rankings oficiales (64%).

Conclusión: La combinación de métricas dinámicas (ELO) con GBDT captura matices del rendimiento que las métricas tradicionales ignoran.

Palabras clave: Machine Learning, XGBoost, ELO Rating, ATP Tennis, Calibración Probabilística, Rolling Stats, FastAPI, Next.js, LLM.

1. Introducción

La predicción de resultados en el tenis ATP enfrenta tres retos: (1) el ranking ATP no captura rachas recientes; (2) un jugador Top 10 puede ser vulnerable en césped frente a un especialista fuera del Top 50; y (3) para análisis de riesgo se requieren probabilidades confiables, no solo etiquetas de clase.

La pregunta de investigación central es: *¿Puede un modelo XGBoost calibrado con ELO dinámico y Rolling Stats predecir el ganador de un partido ATP con un Accuracy superior al 70%, superando el baseline basado en rankings oficiales (64%)?*

El dataset proviene del repositorio `tennis_atp` de Jeff Sackmann, con ~50,000 partidos limpios (2010–2024). Los modelos tradicionales usan regresión logística sobre diferenciales de puntos de ranking; este proyecto adopta Gradient Boosting Decision Trees (GBDT) hibridado con ELO para modelar interacciones no lineales.

3. Arquitectura del Sistema

El sistema sigue una arquitectura de cuatro capas desacopladas. El componente LLM añade explicabilidad semántica a las predicciones del modelo.

Capa	Tecnología	Rol
Data	Python scripts	Descarga y preprocesa CSVs ATP
ML	XGBoost + sklearn	ELO, Rolling Stats, entrenamiento
API	FastAPI/Uvicorn	Endpoints REST + explicabilidad LLM
Frontend	Next.js + shadcn	Interfaz de predicción en tiempo real
Infra	Docker Compose	Orquestación de todos los servicios

Cuadro 2: Componentes del sistema TennisAI.

3.1. Componentes principales

Preprocesamiento y Feature Engineering. Pipeline en `features.py`: (1) eliminación de data leakage, (2) cálculo de ELO dinámico con $K=32$, (3) Rolling Stats con ventana=10 partidos para capturar forma reciente, (4) codificación one-hot de superficie (Hard/Clay/Grass), (5) **Symmetrization:**

cada registro se duplica invirtiendo los jugadores para que el modelo sea agnóstico al orden de entrada. División cronológica 80/20 (train/test).

Modelo de ML. XGBoost seleccionado por su capacidad para manejar datos tabulares heterogéneos y valores faltantes. Parámetros: `learning_rate=0.05`, `n_estimators=200`, `max_depth=5`. **Calibración de Platt** (CalibratedClassifierCV con método sigmoide); los modelos de boosting tienden a dar probabilidades extremas; la calibración garantiza que $p=0.7$ signifique realmente 70% de probabilidad.

Componente LLM. Integrado en la API para generar explicaciones en lenguaje natural de cada predicción, describiendo los factores clave (ELO, superficie, forma reciente) que determinaron el resultado esperado.

API REST. FastAPI expone `/predict`, `/players/search`, `/players/{id}`, `/model/info`. Las predicciones incluyen probabilidades calibradas y explicación LLM.

2. Datos y Exploración (EDA) 2.1.

Descripción del dataset

Atributo	Descripción	Parámetro	Valor
Fuente	github.com/JeffSackmann/tennis_atp	Algoritmo	XGBoost (sklearn API)
Registros (N)	~50,000 partidos tras limpieza	learning_rate	0.05
Features originales	Ranking, edad, altura, mano, superficie, aces, DF, BP	n_estimators	200
		max_depth	5
Features ingenieradas	ELO diff, rolling_avg_ace, rolling_1st_serve	ELO K-factor	32
		Rolling window	partidos
Variable objetivo	target binaria (0=P1 gana, 1=P2 gana)	Calibración	Platt (sigmoide) –
Período	2010–2024		CalibratedClassifierCV
Cuadro 1: Resumen del dataset utilizado.		Framework	scikit-learn + xgboost
		Hardware	CPU / Google Colab (GPU T4)

2.2. Hallazgos del EDA

- **Distribución objetivo:** leve desbalance tras symmetrization

4. Metodología

4.1. Configuración experimental

Cuadro 3: Configuración de entrenamiento.

- (~50/50 por diseño); sin symmetrization, ~58% victorias del mejor rankeado.
- **Features más correlacionadas:** diferencial de ELO y rendimiento reciente en servicio (rolling_avg_ace) son los factores más determinantes según importancia XGBoost.
 - **Valores nulos:** frecuentes en jugadores nuevos (ELO inicial = 1500). XGBoost los maneja nativamente sin imputación.
 - **Especialización por superficie:** el modelo es más preciso en Hard (74%) que en Clay (68%), posiblemente por mayor variabilidad en tierra batida.

4.2. Estrategia de validación

Split cronológico (80% train / 20% test) para simular el escenario real de producción: se predice el futuro con datos del pasado. Este enfoque evita el look-ahead bias que afectaría a un split aleatorio en series temporales deportivas.

4.3. Experimentos realizados

- **Baseline:** regresión logística sobre diferencial de ranking ATP (Accuracy = 64%).
- **Exp. 1:** XGBoost sin calibración, features básicas (Accuracy = 68%).
- **Exp. 2:** XGBoost + ELO + Rolling Stats, sin calibración (Accuracy = 70%).
- **Exp. 3 (mejor):** XGBoost + ELO + Rolling Stats + Calibración de Platt + Symmetrization (Accuracy = 71.2%, Brier Score = 0.191).

6. Discusión

6.1. Interpretación de resultados

El modelo superó el baseline en **+7.2 puntos de accuracy**, confirmando que el ELO dinámico captura mejor la fuerza relativa real del jugador que el ranking ATP (métrica de acumulación anual). La calibración de Platt es indispensable: sin ella, el modelo tiende a probabilidades extremas que no son accionables. El Brier Score de 0.191 indica buena calibración probabilística. La symmetrization elimina el sesgo de posición (quién se llama 'Jugador 1') y mejora la generalización del modelo.

6.2. Limitaciones

- El dataset excluye datos de fatiga (minutos jugados en la última semana), que pueden ser determinantes en torneos Grand Slam

Modelo	Accuracy	Brier Score	Notas
Baseline (Rankings)	64.0%	—	Regresión logística
Exp. 1 – XGB básico	68.0%	0.221	Sin ELO ni calibración
Exp. 2 – XGB + ELO	70.0%	0.208	Sin calibración
Exp. 3 – Completo	71.2%	0.191	Mejor modelo

con partidos consecutivos.

- El ELO se reinicia en 1500 para jugadores nuevos, generando predicciones menos confiables para debutantes en el circuito principal.
- El modelo es menos preciso en Clay (68%) posiblemente por mayor variabilidad de resultados y mayor influencia de factores externos (viento, humedad).
- No se modelan variables meteorológicas, de altitud ni motivación del jugador (e.g., partidos sin consecuencia al final de temporada).

6.3. Trabajo futuro

- Integrar **datos de fatiga** (minutos jugados en la última semana) y variables meteorológicas para mejorar accuracy > 74%.
- Implementar un modelo de **Deep Learning (LSTM)** para capturar secuencias temporales de victorias y detectar rachas y slumps de rendimiento.
- Escalar el componente **LLM** con RAG sobre estadísticas históricas para responder preguntas complejas sobre el historial H2H de los jugadores.

5. Resultados

5.1. Métricas de evaluación

Cuadro 4: Comparación de modelos en test set.

5.2. Importancia de variables

- El **diferencial de ELO** es la feature más determinante, superando al diferencial de ranking ATP como descriptor de la fuerza relativa actual.
- **rolling_avg_ace** captura el rendimiento reciente en servicio y es el segundo factor más importante.
- La **superficie** interactúa significativamente con el ELO: especialistas en clay tienen ELO inflado en esa superficie.

5.3. Ejemplo de output del sistema

```
POST /predict {"player1_id": 104925, "player2_id": 106421, "surface": "Clay"} → {"player1_win_prob": 0.67, "player2_win_prob": 0.33, "predicted_winner": "Djokovic, N.", "elo_diff": 142.3, "explanation": "Djokovic presenta ventaja en ELO (+142) y mejor forma reciente en servicio (rolling_ace: 6.2 vs 4.1)"}
```

Contribuciones del equipo

7. Conclusiones

TennisAI demuestra que la combinación de ingeniería de características específicas del dominio (ELO dinámico, Rolling Stats, Symmetrization)

con algoritmos de ensamble (XGBoost calibrado) permite capturar matices del rendimiento deportivo que las métricas tradicionales ignoran. El sistema alcanzó un **Accuracy de 71.2%** y **Brier Score de 0.191**, superando el baseline de rankings oficiales en **+7.2 puntos**. La calibración de Platt garantiza que las probabilidades sean confiables y accionables. La arquitectura desacoplada (FastAPI + Next.js + Docker) permite escalar el sistema y consumirlo en tiempo real. El componente LLM añade explicabilidad semántica que hace el sistema comprensible para usuarios no técnicos.

Integrante	Contribución principal
Nombre 1	EDA, descarga de datos ATP, cálculo de ELO y Rolling Stats
Nombre 2	Arquitectura XGBoost, calibración de Platt, symmetrization, entrenamiento
Nombre 3	API FastAPI, componente LLM, Docker Compose, integración
Nombre 4	Frontend Next.js, evaluación del modelo, análisis de resultados, informe

GitHub: https://github.com/jmcuaor1/_Atp
Video demo (≤3 min): <https://youtu.be/link-al-video>
Instalación: `pip install -r requirements-api.txt`
Notebook principal: `notebooks/03_modeling.ipynb`

Referencias

[1] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016*. <https://arxiv.org/abs/1603.02754> [2] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.
[3] Sackmann, J. tennis_atp – Datos históricos ATP. https://github.com/JeffSackmann/tennis_atp
[4] Niculescu-Mizil, A. & Caruana, R. (2005). Predicting good probabilities with supervised learning. *ICML 2005*.
[5] FastAPI. (2024). FastAPI framework, high performance, easy to learn. <https://fastapi.tiangolo.com>
[6] Vercel / Next.js. (2024). The React Framework for the Web. <https://nextjs.org>

A. Estructura del proyecto y detalles de hiperparámetros

```
_Atp/ ■■■ backend/ ■■■■ app/ # API FastAPI (routers, servicios) ■■■■ src/ ■■■■ features.py # ELO, Rolling Stats, Symmetrization ■■■■ model.py # Entrenamiento XGBoost + calibración ■■■■ scraper.py # (deprecado) ■■■■ scripts/ # Descarga de datos ATP ■■■■ models/ # tennis_model.pkl, player_profiles.pkl ■■■■ data/raw/ # CSV ATP (2010–2024) ■■■■ frontend/ # Next.js + shadcn/ui ■■■■ notebooks/ # 03_modeling.ipynb ■■■■ docker-compose.yml
```

B. Endpoints principales de la API

Endpoint	Método	Descripción
----------	--------	-------------

/health	GET	Estado del servicio
/predict	POST	Predicción con probabilidad calibrada y explicación LLM
/players/search	GET	Buscar jugador por nombre (?q=)
/players/{id}	GET	Perfil completo + ELO actual del jugador
/model/info	GET	Métricas, Brier Score y metadatos del modelo